The Analog Coding Workbook

Python Edition

Jeremiah Carey

First Edition

Copyright © 2026 Jeremiah Carey. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Published independently.

First edition, 2026

For James and Elena.

Table of Contents

Introduction 5
Part 1 — Warmups 11
Solutions 20
Warmup Answers 21
Appendix 23

Introduction

The Changing Nature of Programming

Programming today looks very different than it did even a few years ago. Modern developers work in environments filled with powerful tools: intelligent editors, autocomplete systems, vast online documentation, reusable libraries, and increasingly capable AI assistants. Many of these tools dramatically increase productivity. Code that once took hours to write can now be produced in minutes. Entire functions, classes, or even applications can be generated with a few prompts.

These changes are not necessarily a problem (or so I would argue). In fact, they represent real progress. Better tools allow developers to build more ambitious systems, reduce repetitive work, and focus on higher-level design. The goal of this book is not to argue against these tools. But as tools change, the way skills develop and are maintained changes as well.

There is a difference between producing code and understanding code. There is a difference between recognizing a solution and being able to construct it. There is a difference between guiding a tool and doing the work yourself. Modern programming environments often blur these distinctions. It is possible to be highly productive while relying heavily on autocomplete, or generated code. It is possible to spend hours working with code without ever forcing yourself to recall syntax, reason through logic step by step, or hold a complete solution in your head.

This shift has consequences. Like any skill, programming ability depends on practice. When the nature of practice changes, the nature of skill changes with it. The purpose of this workbook is to provide a form of practice that modern workflows rarely require, but that remains essential for long-term mastery. The tools have changed. That means the way we practice and learn has to change too.

Modern Tools and Skill Decay

Most developers have had the same unsettling experience at some point. You sit down in front of a blank page (or even a blank screen) — perhaps in an interview, perhaps on a whiteboard, perhaps in a notebook — and realize that writing code from scratch feels harder than it used to. You know the basic concepts. You recognize the patterns. You could solve the problem quickly with an editor open. But without autocomplete or an

AI agent close to hand, without running the code, without searching for examples, the details are harder to recall than expected.

This does not mean you have become a worse programmer. In some sense, the opposite is true. Modern developers often work on larger systems, use more sophisticated tools, and solve more complex problems than ever before. But the environment in which we work has changed. We spend less time writing code line by line, and more time navigating, editing, reviewing, and assembling code with the help of powerful software. And when the environment removes friction, practice becomes less automatic.

In earlier eras, writing code required constant recall. You had to remember method names, syntax rules, and control structures because the editor could not supply them for you. You had to reason carefully about logic because running the program was slower and more cumbersome. Today, the cost of trial and error is much lower. If something does not work, you can run the code, check the error, search for an example, or ask an AI assistant. These are all useful abilities, for which we should be grateful, but they do not train the same mental muscles as writing a correct solution from memory.

Many developers write thousands of lines of code each week, but think through very few. Over time, this can lead to a subtle form of skill decay. You may still be productive, but less precise. You may still solve problems, but rely more on tools to fill in the gaps. You may still understand code when you see it, but find it harder to produce it yourself without assistance. In situations where tools are limited — interviews, debugging sessions, design discussions, teaching, or reviewing unfamiliar code — the difference becomes noticeable.

This workbook is designed to address that gap, not by replacing modern tools, but by allowing them to be balanced with deliberate practice. Again, the point is not that modern tools should be regretted, but that there are good reasons to make sure a baseline of old-school skill is maintained.

Why Write Code By Hand

In many fields, practicing simple patterns or without tools is considered normal. Musicians practice scales without accompaniment. Athletes drill fundamentals even after mastering advanced techniques. Mathematicians work problems on paper long after calculators become available. Writers sketch outlines by hand before typing final drafts. In each case, the goal is not efficiency, but a deeper understanding.

Research in education and cognitive science has repeatedly shown that writing by hand engages the brain differently than typing. Handwriting is slower, and that slowness

forces deeper processing. When you write something yourself, you cannot rely on recognition alone; you must recall information from memory and organize it consciously. This leads to better retention, stronger understanding, and more durable skills.

Typing, by contrast, often encourages a more passive style of work. You can begin writing before you fully know what you want to say. You can correct mistakes instantly. You can rely on suggestions, autocomplete, or copy-and-paste. These features are extremely useful for real work, but they reduce the amount of effort required to produce each line. Effort is not always a disadvantage — in practice effort is the primary catalyst for learning!

Writing code by hand forces you to slow down. You must think about syntax before you write it. You must plan the structure of a solution before committing it to the page. You cannot run the program to see what happens; you must reason through it step by step. You cannot depend on the editor to remind you of method names or argument order; you must remember them yourself. This provides a deeper and more durable understanding.

The Lack of *Practice* in Professional Coding

Modern development environments are designed to reduce friction, and for real projects this is exactly what we want. Editors complete our code. Linters fix small mistakes. Tests catch errors. Documentation is always a few clicks away. AI assistants can generate entire solutions in seconds. These tools allow us to move faster and build more.

However, they also change the kind of practice we get. A typical work session today might involve reading existing code, modifying a few lines, running tests, searching for examples, copying patterns from other files, and asking an AI tool to generate a function that you then edit. This is productive work, but it does not require the same level of recall and reasoning as writing a solution from scratch.

Work is not the same as practice. You can spend an entire day coding without ever forcing yourself to remember the exact syntax of a loop, to design a function without hints, or to trace the behavior of code entirely in your head. Over time, the brain adapts to the environment it is given. If the environment always provides assistance, the ability to work without assistance weakens.

Again: work and practice are distinct. A musician performing a concert is not practicing; they are executing. Practice happens when you isolate a difficult passage

and play it until the movement becomes automatic. Modern coding tools are designed to help you produce output, but they are not designed to build skill. If you want to stay sharp, practice must be added intentionally in a context that doesn't let the tools cover for you.

About this Workbook

This book is a collection of exercises designed for deliberate practice. It is not a tutorial, and it is not meant to teach the language from the beginning. It assumes that you already know the basics and want to strengthen your ability to use them confidently. The goal is not to memorize trivia, but to improve fluency — the ability to read, write, and reason about code without depending on an editor or external reference.

Each problem in this workbook is meant to be solved with pencil and paper. Do not run the code while you are working. Do not open an editor. Do not search for examples. Try to complete the solution using only your own understanding.

Some problems ask you to write code. Others ask you to predict output, trace execution, find bugs, or reason about behavior. These are all skills that working developers use every day, but often with the support of powerful tools. Here, the tools are removed so that the thinking becomes visible.

This kind of practice can feel slower, and sometimes frustrating. That is part of the point. The difficulty you feel when working without assistance is exactly the signal that the exercise is doing its job.

On the other hand, if you are more advanced, many of the problems here may feel a bit too easy. That's ok, too. Think again of the scales analogy. For a musician of any experience, scales are not difficult or technically challenging. But they're still important. They help encode certain patterns in our brain, make certain techniques second nature, and make sure the fundamentals are not lost sight of.

The problems span five types:

- **Warmups** — short snippets where you predict the output. These are low-stakes, quick, and designed to get your brain engaged. They cover the kinds of behavior that are easy to misread: loops, scope, mutation, reference types.
- **Write the Code** — the core and longest section. You're given a prompt and a blank graph paper area. You write a working function by hand. No starter code, no hints unless provided. The problems range from simple to hard, but none of them should require memorizing algorithms or specialized study.

- **Trace the Program** — you're given a complete piece of code and asked to simulate its execution step by step, tracking what variables hold and what the function returns. This is harder than it looks, and more revealing.
- **Find the Bug** — you're given code with a single bug and asked to identify it, explain it, and write the fix. The bugs all represent the sorts of issues developers face all the time: off-by-one errors, wrong operators, missing returns, accidental mutations, the kinds of mistakes that appear in production code.
- **Challenge Problems** — harder problems in the write-the-code format. Still no algorithms, still solvable by a working developer — but multi-step, with more room for error.

Don't feel that you need to work straight from the beginning all the way through in order. Feel free to jump around as you see fit, or set a daily discipline of doing 1–3 problems before you start your work day.

There is an answer key for each of these sections in the back. With most coding problems, there are multiple "correct" answers, so if the answer you've written doesn't exactly match that in the answer key, use that as an excuse to understand the differences, verify that your code works, and think about why one answer might be preferable to another.

There is also a very brief appendix in the back covering standard Python syntax and built-in methods. Just enough to jog your memory if you forget how to write a while loop or how to use the `sorted()` function.

Who This Book is For

This workbook is intended for programmers who already know the basics of the language and want to strengthen their fluency. This book is not a replacement for real projects, and it is not meant to teach programming from the beginning. It is a supplement — a way to practice the core skills that underlie all programming, but that modern workflows do not always exercise.

Those who may find this book useful include:

- Working developers who want to stay sharp.
- Students who are learning Python and want practice that goes beyond tutorials.
- Self-taught programmers who feel strong in some areas but patchy in others.
- Developers preparing for interviews who want to practice writing code without a safety net.

- Anyone who has spent the last year mostly reading, reviewing, and modifying code rather than writing it.
- Anyone who has started to rely on autocomplete more than they'd like.
- Anyone who feels a little uncomfortable on a whiteboard.

Analog Coding with Python

In photography, analog cameras require more attention than digital ones, but they often lead to a deeper understanding of exposure and composition. In music, practicing unplugged can reveal weaknesses that are hidden when amplification is available. In writing, drafting by hand forces a slower, more deliberate style of thinking.

Programming is no different. Modern tools make development faster and more comfortable, but they also hide some of the effort that builds skill. Working with pencil and paper removes those conveniences and makes the thinking visible again. This is what this book calls analog coding — writing code slowly, deliberately, and without assistance, in order to strengthen the habits that make all other work easier. This volume focuses on Python fundamentals, including lists, dictionaries, functions, and common patterns used in everyday development.

The best way to understand the value of this kind of practice is to try it. Turn the page, pick up a pencil, and begin.

Part 1 — Warmups

Predict the output of each program without running the code.

1. Reassignment

variables

```
x = 5
x = x + 2
x = x * 3

print(x)
```

Output:

2. Loop Boundary

loops

```
total = 0

for i in range(4):
    total += i

print(total)
```

Output:

3. List Mutation

lists

```
items = [1, 2, 3]
items[1] = 99

print(items)
```

Output:

4. Function Return

functions

```
def double(n):
    return n * 2

result = double(4)
print(result)
```

Output:

5. The Greeting

functions

```
def greet(name):
    message = "Hello, " + name

result = greet("Maya")
print(result)
```

Output:

6. Two Variables

variables

```
a = 3
b = a
a = 8

print(a)
print(b)
```

Output:

7. Dict Mutation

dictionaries

```
user = {"name": "Ava", "age": 30}
user["age"] = user["age"] + 1

print(user["age"])
```

Output:

8. Shared List

references

```
a = [1, 2]
b = a
b.append(3)

print(a)
print(b)
```

Output:

9. Going Independent

references

```
a = [1, 2]
b = a[:]
b.append(3)

print(a)
print(b)
```

Output:

10. Truthiness

truthiness

```
print(bool([]))
print(bool([0]))
print(bool(""))
print(bool("hi"))
```

Output:

11. Equality vs Identity

equality

```
a = [1, 2]
b = [1, 2]
c = a

print(a == b)
print(a is b)
print(a is c)
```

Output:

12. Division

operators

```
print(5 / 2)
print(5 // 2)
```

Output:

13. append + len

lists

```
names = ["Ana", "Ben"]
names.append("Chris")

print(len(names))
print(names)
```

Output:

14. List Comprehension

comprehensions

```
nums = [1, 2, 3, 4]
doubled = [n * 2 for n in nums]

print(doubled)
```

Output:

15. Filtering with a Comprehension

comprehensions

```
nums = [1, 2, 3, 4, 5, 6]
evens = [n for n in nums if n % 2 == 0]

print(len(evens))
print(evens)
```

Output:

Solutions

Warmup Answers

1. Reassignment

Answer:
```
21
```

2. Loop Boundary

Answer:
```
6
```

3. List Mutation

Answer:
```
[1, 99, 3]
```

4. Function Return

Answer:
```
8
```

5. The Greeting

Answer:
```
None
```

6. Two Variables

Answer:
```
8
3
```

7. Dict Mutation

Answer:
```
31
```

8. Shared List

Answer:
```
[1, 2, 3]
[1, 2, 3]
```

9. Going Independent

Answer:
```
[1, 2]
[1, 2, 3]
```

10. Truthiness

Answer:
```
False
True
False
True
```

11. Equality vs Identity

Answer:
```
True
False
True
```

12. Division

Answer:
```
2.5
2
```

13. append + len

Answer:
```
3
['Ana', 'Ben', 'Chris']
```

14. List Comprehension

Answer:
```
[2, 4, 6, 8]
```

15. Filtering with a Comprehension

Answer:
```
3
[2, 4, 6]
```

Appendix

Appendix

This appendix contains a brief reference for common Python syntax and methods. It is not a full guide to the language — only the most commonly needed features for solving the problems in this workbook are included.

Function Syntax

Function definition

```
def add(a, b):
    return a + b
```

Default parameters

```
def greet(name="Guest"):
    return "Hello, " + name
```

Lambda (anonymous function)

```
add = lambda a, b: a + b
```

Conditionals and Operators

if / elif / else

```
if x > 10:
    print("big")
elif x > 5:
    print("medium")
else:
    print("small")
```

Comparison operators

```
==    equal
!=    not equal
>     greater than
<     less than
>=    greater or equal
<=    less or equal
```

Logical operators

```
and   both conditions true
or    at least one true
not   negation
```

Example:

```
if age > 18 and is_member:
    pass
```

Ternary expression

```
result = "big" if x > 10 else "small"
```

Falsy values

```
False
0
0.0
""
None
[]
{}
```

Everything else is truthy.

Loop Syntax

for loop (range)

```
for i in range(5):
    print(i)
```

for loop (list)

```
for value in my_list:
    print(value)
```

while loop

```
i = 0
while i < 5:
    print(i)
    i += 1
```

enumerate (index + value)

```python
for i, value in enumerate(my_list):
    print(i, value)
```

break / continue

```python
for x in my_list:
    if x == 5:
        break

for x in my_list:
    if x < 0:
        continue
    print(x)
```

List Methods

Adding / removing

```python
lst.append(x)       # mutates — adds to end
lst.pop()           # mutates — removes and returns last item
lst.pop(i)          # mutates — removes and returns item at index i
lst.insert(i, x)    # mutates — inserts x at index i
lst.remove(x)       # mutates — removes first occurrence of x
```

Searching

```python
x in lst            # returns boolean
lst.index(x)        # returns index, raises ValueError if not found
lst.count(x)        # returns number of occurrences
```

Copying / slicing

```python
lst[start:end]      # new list — does not mutate
lst[:]              # full copy
lst.copy()          # full copy
```

Sorting

`sort()` **mutates the original list**. `sorted()` returns a new list.

```python
lst.sort()                        # mutates — ascending
lst.sort(reverse=True)            # mutates — descending
lst.sort(key=lambda x: x[1])     # mutates — custom key

sorted(lst)                       # returns new list — does not mutate
```

```
sorted(lst, reverse=True)
sorted(lst, key=lambda x: x[1])
```

Other

```
len(lst)            # length
sum(lst)            # sum of numeric items
min(lst)            # minimum
max(lst)            # maximum
lst.reverse()       # mutates — reverses in place
list(reversed(lst)) # returns new reversed list
```

List comprehensions

```
squares = [x * x for x in range(10)]

evens = [x for x in lst if x % 2 == 0]
```

String Methods

Length

```
len(s)
```

Searching

```
"a" in s
s.startswith("a")
s.endswith("a")
s.find("a")         # returns index, or -1 if not found
s.index("a")        # returns index, raises ValueError if not found
```

Slicing

```
s[1:4]
s[:3]
s[-1]               # last character
```

Case

```
s.lower()
s.upper()
```

Trimming

```
s.strip()
s.lstrip()
s.rstrip()
```

Splitting / joining

```
s.split(",")
",".join(lst)
```

Replace

```
s.replace("a", "b")
```

Check content

```
s.isdigit()
s.isalpha()
s.isalnum()
```

Dictionary Syntax and Methods

Dictionary literal

```
user = {
    "name": "Alice",
    "age": 30
}
```

Accessing values

```
user["name"]
user.get("name")           # returns None if key missing
user.get("name", "Guest")  # returns default if key missing
```

Adding / updating

```
user["age"] = 31
user["city"] = "NY"
```

Checking for a key

```
if "name" in user:
    pass
```

Keys / values / items

```
user.keys()
user.values()
user.items()
```

Iterating:

```
for key, value in user.items():
    print(key, value)
```

Removing

```
del user["age"]
user.pop("age")             # removes and returns value
user.pop("age", None)       # safe — returns None if missing
```

Dictionary comprehension

```
squares = {x: x * x for x in range(5)}
```

Useful Built-in Functions

```
range(n)            # 0 to n-1
range(a, b)         # a to b-1
range(a, b, step)   # with step

abs(x)              # absolute value
round(x, n)         # round to n decimal places

type(x)             # type of x
isinstance(x, int)  # type check

zip(a, b)           # pair up two iterables
map(fn, lst)        # apply function to each item — returns iterator
filter(fn, lst)     # filter items — returns iterator
any(lst)            # True if any item is truthy
all(lst)            # True if all items are truthy
```

Common Patterns

Swap two variables

```
a, b = b, a
```

Unpack a list

```
first, *rest = lst
*rest, last = lst
first, *middle, last = lst
```

Count occurrences with a dict

```
counts = {}
for item in lst:
    counts[item] = counts.get(item, 0) + 1
```

Flatten a list of lists

```
flat = [x for sublist in nested for x in sublist]
```

Check if a number is even/odd

```
x % 2 == 0   # even
x % 2 != 0   # odd
```